

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 6

Templates und Routing

6.1 Template-Systeme

Template-Systeme dienen zur Entkopplung von HTML- und PHP-Code. PHP bietet in seiner Basisinstallation kein System an, kann jedoch streng gesehen selbst als Template-System gesehen werden, da statischem HTML-Markup mit PHP-Code dynamische Inhalte hinzugefügt werden können. Da diese Vermischung jedoch meist unübersichtlich ist und auch durchaus noch mehr Funktionen haben könnte bzw. sollte, haben sich diverse Libraries am Markt etabliert.

6.1.1 Allgemeines

Die Trennung einzelner Teile einer Webseite kann zu einer besseren Übersicht und leichter Wartbarkeit beitragen. Durch die Trennung von HTML und CSS in separate Dateien muss der Markup-Code nicht verändert werden, wenn irgendwo eine Farbe ausgetauscht werden soll. Durch die Trennung von HTML und JavaScript kann die lokale Programmlogik ebenfalls in einer eigenen Datei implementiert werden, ohne dazu jedes Mal das HTML-Gerüst editieren zu müssen.

Auch für PHP gilt diese Anforderung, wenngleich sie bis jetzt nicht umgesetzt wurde, denn alle Beispiele bisher haben Logik und Ausgabe (also PHP- und HTML-Code) miteinander vermischt. Die Befehle `echo` und `print()` im PHP-Code sind dafür hauptverantwortlich, erzeugen sie doch HTML-Code mitten in PHP-Skripten. Sollen einzelne HTML-Teile wiederverwendet werden, müssen diese in eine Funktion oder Methode ausgelagert werden. Techniken wie Advanced Escaping, bei denen zwischen logischen PHP-Anweisungen (z. B. `for`-Schleifen) und erzeugter HTML-Ausgabe hin und her gewechselt werden, oder die Heredoc-Syntax komplettieren die Vermischung von Code und Struktur, die aber eigentlich nicht gewünscht ist.

Template-Systeme schaffen hierbei Abhilfe. Konstante HTML-Teile werden in eigene Dateien ausgelagert und somit vom PHP-Code entfernt. Dynamische Teile, die aufgrund der Programmlogik unterschiedlich sein können, werden durch Platzhalter aus PHP entsprechend befüllt und ergeben somit wieder eine dynamische Seite. Dies erlaubt zum einen ein konsistentes Seitendesign, auf der anderen Seite eine einfachere Wartbarkeit und größere Übersicht.

6.1.2 Bekannte Template-Engines

Es gibt viele unterschiedliche Template-Systeme für PHP. Viele davon sind als eigenständige Bibliotheken verfügbar:

- Latte¹,
- Twig²,
- Smarty³,
- mustache⁴,
- Plates⁵.

Andere Template-Systeme sind Teil von PHP-Frameworks:

- Blade (Teil des Laravel Frameworks)⁶,
- Volt (Teil des phalcon Frameworks)⁷.

Die Auswahl des Template-Systems ist vielfach persönliche Präferenz oder hängt vom genauen Einsatzgebiet ab. Das Erlernen eines Systems ist jedoch in der Regel ein guter Grundstein, um auch einen schnellen Einstieg in andere Template-Engines zu finden. Im Folgenden wird Latte vorgestellt, ein intuitives, sicheres und sehr modernes Template-System.

6.1.3 Latte Template Engine

In den letzten Jahren hat sich *Latte* als extrem sichere und performante Template-Engine einen Namen gemacht. Latte zeichnet sich besonders durch sein „Context-Aware Escaping“ aus. Das bedeutet, die Engine erkennt automatisch, ob eine Variable in einem HTML-Text, einem HTML-Attribut oder gar in einem JavaScript-Block ausgegeben wird und wendet selbstständig die korrekten Sicherheitsmechanismen an, um Cross-Site Scripting (XSS, siehe Vorlesung 7) zu verhindern.

Die Syntax von Latte orientiert sich stark an nativem PHP, was den Einstieg deutlich erleichtert. Die aktuelle Version ist 3.1.4 vom 23. April 2026.

Installation

Latte ist eine moderne und modular aufgebaute PHP-Anwendung und besteht aus über 170 PHP-Dateien (Klassen, Interfaces etc.). Damit dieses System funktioniert, spielt Autoloading eine zentrale Rolle, damit die Klassen untereinander gefunden werden. Die bevorzugte Installationsweise⁸ für Latte ist daher mittels Composer, da dieser das Autoloading für Komponenten bereits beinhaltet.

```
1 $ composer require latte/latte
```

¹<https://latte.nette.org/>

²<https://twig.symfony.com/>

³<https://www.smarty.net/>

⁴<https://mustache.github.io/>

⁵<https://platesphp.com/>

⁶<https://laravel.com/docs/master/blade>

⁷<https://docs.phalcon.io/latest/volt/>

⁸<https://latte.nette.org/en/develop#toc-installation>

Funktionsweise und Konfiguration

HTML-Inhalte werden bei Template-Systemen in Dateien ausgelagert. Bei Latte haben diese üblicherweise die Endung `.latte`. Um diese in PhpStorm sinnvoll verwenden zu können, empfiehlt sich das Plugin *Latte Support*⁹.

Ein Template ist zunächst wie eine statische HTML-Datei. Es gibt jedoch Platzhalter für Variablen, die von PHP aus beim Rendern des Templates befüllt werden. Somit bleibt die dynamische Natur, die auch bei der nativen Ausgabe mit PHP vorherrscht, erhalten.

Um Templates mit Latte anzuzeigen, muss ein zentrales Engine-Objekt angelegt werden. Es ist empfehlenswert, ein Verzeichnis für den Cache anzugeben (hier `cache`, da Latte die Templates für maximale Performance im Hintergrund in reinen PHP-Code kompiliert. Während der Entwicklung erkennt Latte Änderungen an den `.latte`-Dateien automatisch und kompiliert diese bei Bedarf neu. Weiters kann noch explizit ein `FileLoader` mit dem Pfad der Template-Dateien angegeben werden. Dies schränkt einerseits das Laden von Template-Dateien auf ein Verzeichnis ein (hier `templates`) und erspart die Angabe des gesamten Verzeichnispfades.

Der initiale Konfigurationsaufwand sieht wie folgt aus:

```
1 use Latte\Engine;
2 use Latte\Loaders\FileLoader;
3
4 $latte = new Engine();
5 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
6 $latte->setCacheDirectory(__DIR__ . "/cache");
```

Ausgabe, Variablen und Zuweisung

Latte verfügt über die Methode `render($name, $params)`¹⁰, um das Template anzuzeigen. Der erste Parameter ist der Pfad zum anzuzeigenden Template, der zweite ein Array oder Objekt mit den zu übergebenden Parametern. Soll das Template nur gerendert, d. h. dessen HTML-Code erzeugt und als String zurückgegeben werden, kann `renderToString($name, $params)` verwendet werden. Der folgende Aufruf zeigt das Template `message.latte` an und weist die Parameter `name` und `message` im Template zu:

```
1 $latte->render("message.latte", [
2     "name" => "John Doe",
3     "message" => "I'm back baby!",
4 ]);
```

Damit die angegebenen Parameter im Template auch angezeigt werden, müssen Platzhalter gesetzt werden. Latte nutzt dafür einfache geschwungene Klammern mit dem Variablennamen (inklusive dem Dollar-Zeichen, genau wie die Curly Syntax in PHP):

```
1 <p>Name: {$name}</p>
```

⁹<https://plugins.jetbrains.com/plugin/24218-latte-support>

¹⁰<https://latte.nette.org/en/develop#toc-how-to-render-a-template>

Beispiel

Dies alles wird in einem Beispiel demonstriert. Die Datei `message.latte` enthält das HTML-Template, die Datei `latte.php` nimmt die Konfiguration vor und zeigt das Template an.

Das Template enthält die beiden Platzhalter `{ $name }` und `{ $message }`:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Message</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <p>Name: { $name }</p>
9     <p>Message: { $message }</p>
10  </body>
11 </html>
```

Die PHP-Datei initialisiert die Engine und zeigt das Template an:

```
1 use Latte\Engine;
2 use Latte\Loaders\FileLoader;
3
4 $latte = new Engine();
5 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
6 $latte->setCacheDirectory(__DIR__ . "/cache");
7
8 $latte->render("message.latte", [
9   "name" => "John Doe",
10  "message" => "I'm back baby!",
11 ]);
```

Die Ausgabe ist das folgende HTML-Markup, das Ergebnis ist in Abbildung 6.1 ersichtlich.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Message</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <p>Name: John Doe</p>
9     <p>Message: I'm back baby!</p>
10  </body>
11 </html>
```

Fortgeschrittene Variablenübergabe

In Latte können zusammengesetzte Datentypen exakt wie in nativem PHP abgefragt werden.

Arrays und Objekte: Der Zugriff auf Arrays im Template ist analog zu PHP möglich:

```
1 $array = [
2   "John Doe",
```

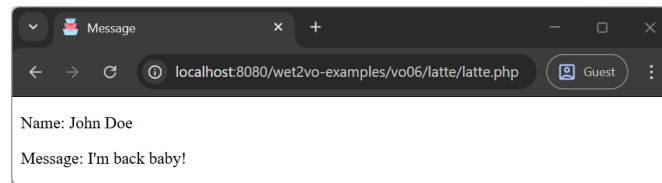


Abbildung 6.1: Bei der Ausgabe der Seite ist nicht ersichtlich, dass Latte im Hintergrund arbeitet. Aufgerufen wird ja nach wie vor die PHP-Seite (hier `latte.php`).

```

3  "male",
4  25,
5 ];
6 $latte->render("array-object.latte", [
7  "data1" => $array,
8 ]);

```

In der Template-Datei kann dann über die gewohnte Syntax auf die einzelnen Stellen zugegriffen werden:

```

1 <p>Name: {{$data1[0]}}</p>
2 <p>Gender: {{$data1[1]}}</p>
3 <p>Age: {{$data1[2]}}</p>

```

Auch bei assoziativen Arrays und Objekten bleibt die Syntax konsistent. Folgendes Array soll ans Template übergeben werden:

```

1 $assocArray = [
2  "name" => "Jane Doe",
3  "details" => [
4    "gender" => "female",
5    "age" => 23,
6  ],
7 ];
8 $latte->render("array-object.latte", [
9  "data2" => $assocArray,
10 ]);

```

Der Zugriff im Template:

```

1 <p>Name: {{$data2["name"]}}</p>
2 <p>Gender: {{$data2["details"]["gender2"]}}</p>
3 <p>Age: {{$data2["details"]["age"]}}</p>

```

Auch Objekte können an Latte-Templates übergeben werden. Auf die Eigenschaften eines Objekts wird über den `->`-Operator zugegriffen. Zunächst wird ein Objekt angelegt und übergeben:

```

1 $object = new Person(
2  "Jim Doe",
3  "male",
4  3,
5 );
6 $latte->render("array-object.latte", [
7  "data3" => $object,
8 ]);

```

Im Template wird auf die einzelnen (public)-Eigenschaften wie folgt zugegriffen:

```

1 <p>Name: {$data3->name}</p>
2 <p>Gender: {$data3->gender}</p>
3 <p>Age: {$data3->age}</p>

```

Filter: Um zugewiesene Variablen im Template noch weiter zu modifizieren, existieren *Filter*. Diese werden mit dem |-Zeichen (Pipe) an die Variable gehängt. Dabei können auch mehrere Filter angewandt und Parameter an den Filter übergeben werden.

Latte bietet eine Vielzahl eingebauter Filter¹¹. Einige davon sind:

- Groß-/Kleinschreibung: `capitalize`, `upper`, `lower`, `firstUpper`, `firstLower`,
- Transformation: `stripHtml`, `breakLines`, `trim`, `sort`, `date`,
- Escaping: `noescape`, `escapeUrl`, `query`,
- ...

Das folgende Beispiel formatiert einige Werte:

```

1 <p>Name: {$name|lower}</p>
2 <p>Message: {$message|stripHtml|trim}</p>
3 <p>Timestamp: {=now|date:'d.m.Y H:M'}</p>

```

Hinweis: Latte erlaubt das direkte Aufrufen von PHP-Funktionen (wie `date()`) innerhalb der Tags `{}`: `{date("d.m.Y, H:i:s")}`

Tags und Kontrollstrukturen

Mit Latte lässt sich Logik im Template hervorragend abbilden. Auch hier orientieren sich die Tags¹² an PHP.

Schleifen: Mit `{foreach}`¹³ lassen sich Schleifen über komplette Arrays realisieren.

```

1 <ul>
2   {foreach $colors as $color}
3     <li>{$color}</li>
4   {/foreach}
5 </ul>

```

Anstatt Tags zu verwenden, kann auch die *n:attributes*-Syntax verwendet werden. Dabei wird die Logik als HTML-Attribut verpackt. Latte erkennt dies beim Kompilieren und wandelt es in korrektes HTML um. Eine `foreach`-Schleife sieht damit wie folgt aus:

```

1 <ul>
2   <li n:foreach="$colors as $color">{$color}</li>
3 </ul>

```

Auch eine klassische `for`-Schleife mit einer Zählervariable kann in Latte verwendet werden:

```

1 <ul>
2   {for $i = 1; $i <= 3; $i++}
3     <li>{$i}</li>
4   {/for}
5 </ul>

```

¹¹<https://latte.nette.org/en/filters>

¹²<https://latte.nette.org/de/tags>

¹³<https://latte.nette.org/en/tags#toc-loops>

Auch diese Schleife kann als *n:attribute* formuliert werden:

```
1 <ul>
2   <li n:for="$i = 1; $i <= 3; $i++">{$i}</li>
3 </ul>
```

Innerhalb einer `foreach`-Schleife stellt Latte die implizite Variable `$iterator` zur Verfügung, mit der sich etwa abfragen lässt, ob es der erste (`$iterator->first`) oder letzte (`$iterator->last`) Durchlauf ist.

Bedingungen: Bedingungen werden mit `if`, `elseif` und `else` abgebildet.

```
1 {if $gender === "male"}
2   <p>Welcome Sir!</p>
3 {elseif $gender === "female"}
4   <p>Welcome Madam!</p>
5 {else}
6   <p>Welcome non-binary Person!</p>
7 {/if}
```

Mit `n:if` und `n:else` existieren für einfache Bedingungen auch *n:attributes*. Ein `n:elseif` gibt es jedoch nicht, weshalb das obige Beispiel nur mit Tags abgebildet werden kann.

Soll überprüft werden, ob eine Variable existiert (d. h. ins Template übergeben wurde), bietet Latte den speziellen `ifset`-Tag:

```
1 {ifset $name}
2   <p>Welcome {$name}!</p>
3 {/ifset}
```

Diese kann wiederum kürzer formuliert werden:

```
1 <p n:ifset="$name">Welcome {$name}!</p>
```

Weitere Templates inkludieren: Um Templates modular aufzubauen (z. B. Header und Footer auszulagern), nutzt Latte die `include`-Anweisung¹⁴.

```
1 {include "filename.latte"}
```

Zusätzlich können beliebige Variablen direkt bei der Einbindung übergeben werden. Parameter werden durch Kommas getrennt und mit dem `=>` Operator zugewiesen:

```
1 // main.latte
2 <!DOCTYPE html>
3 <html lang="en">
4   <head>
5     <title>Hello World</title>
6   </head>
7   <body>
8     {include "body.latte", heading => "Welcome to the site!"}
9   </body>
10 </html>
```

```
1 // body.latte
2 <h1>{$heading}</h1>
3 <p>Hello World!</p>
```

¹⁴<https://latte.nette.org/en/tags#toc-include>

Latte bietet noch weitere Funktionalitäten und lässt sich auch exzellent erweitern. All diese Dinge finden sich in der offiziellen Latte Dokumentation [2].

6.2 Routing

Routing im Kontext der Webentwicklung ist ein Mechanismus, bei dem HTTP-Requests, d. h. der Aufruf von bestimmten URLs, an den jeweils definierten Code, der sich um deren Abarbeitung kümmert, weitergeleitet werden. Einfach ausgedrückt bedeutet dies, dass Routing bestimmt, was passiert, wenn Nutzer*innen bestimmte URLs aufrufen.

6.2.1 Allgemeines

In seiner grundsätzlichen Konfiguration verlässt sich PHP beim Routing auf den Webserver. Dabei wird der Pfad des Requests, also der Teil nach der Domain oder IP-Adresse, direkt auf ein Verzeichnis am Webserver und die darin enthaltenen Dateien abgebildet.

Beim URL `https://www.example.com/mywebapp/index.php` ergeht der Request an den Server, der unter `www.example.com` verfügbar ist. Der Webserver sucht danach im Ordner `mywebapp` eine Datei `index.php`, übergibt diese an die PHP-Engine und nach Ausführung des Inhalts wird die HTML-Antwort (Response) zurückgegeben. Genau so läuft diese auf einem Entwicklungsserver ab: `http://localhost:8080/wet2vo-examples/vo06/routing/index.php` sendet den Request an den Server unter `localhost` am Port 8080. Dort muss – ausgehend vom Web-Root, das üblicherweise `var/www/html` lautet) – die Verzeichnishierarchie `wet2vo-examples/vo06/routing` existieren und darin die Datei `index.php` liegen, damit die Datei ausgeführt und eine Antwort zurückgegeben werden kann. Existiert unter dem angegebenen Pfad keine Datei, so liefert der Webserver die Antwort `404: Not found`, da die Ressource nicht vorhanden war.

Dieser Mechanismus ist einfach und trägt nicht zuletzt auch zum Erfolg von PHP bei. Dieses Prinzip ist schnell erlernt und verstanden, der Webserver muss dabei nicht aufwändig konfiguriert werden. Wurde eine PHP-Datei erstellt, kann sie am Webserver in den gewünschten Ordner gelegt werden und ist darunter sofort verfügbar.

Diese Einfachheit hat jedoch auch den Nachteil, dass sich der URL und die Verzeichnisstruktur immer decken müssen. Zudem hört jeder URL zwangsläufig mit einer PHP-Datei auf, denn diese enthält ja den Code, der aufgerufen werden muss. Eine Webseite, die also aus Hauptseite, Registrierung, Login und Logout besteht, benötigt die folgenden PHP-Dateien, die etwa unter diesen URLs aufgerufen werden können:

- `https://www.example.com/mywebapp/index.php`: Hauptseite
- `https://www.example.com/mywebapp/register.php`: Registrierung
- `https://www.example.com/mywebapp/login.php`: Login
- `https://www.example.com/mywebapp/logout.php`: Logout

Nehmen wir jedoch an, es besteht nun die Anforderung, folgende URLs (genau in dieser Schreibweise) zu erreichen:

- `https://www.example.com/mywebapp/`: Hauptseite
- `https://www.example.com/mywebapp/account/register`: Registrierung
- `https://www.example.com/mywebapp/account/login`: Login
- `https://www.example.com/mywebapp/account/logout`: Logout

Der vielleicht zunächst in den Sinn kommende Ansatz wäre, im Ordner `mywebapp` den Unterordner `account` anzulegen und darin wiederum `register`, `login` und `logout`. Die PHP-Dateien im Root-Ordner und den Unterordnern müssten dann jeweils `index.php` heißen, somit müssten sie nicht angegeben werden, denn ein Webserver sucht bei einer reinen Verzeichnisangabe immer nach Dateien mit dem Namen `index` und verschiedenen Endungen und ruft diese automatisch auf, wenn er sie findet.

Dieses Prinzip funktioniert hier zwar, wird jedoch schnell umständlich. Denn für jede Pfadebene müsste tatsächlich am Webserver ein Verzeichnis angelegt werden, darüber hinaus funktioniert ein URL wie `https://www.example.com/mywebapp/index.php` dann auch und das ist eigentlich nicht gewünscht. Und dann gibt es womöglich auch noch URLs, deren genauer Inhalt gar nicht bekannt ist. Soll etwa ein Produkt mit einer bestimmten ID abgefragt werden, so könnte die URL `https://www.example.com/mywebapp/product/4` lauten, um das Produkt mit der ID 4 anzuzeigen. Auch hier bräuchte es für jede ID ein Verzeichnis – wenn Produkte dynamisch aus einer Datenbank kommen, ein nicht mehr wartbares Unterfangen.

Es benötigt also Routing, welches unabhängig von einer physischen Verzeichnisstruktur am Server funktioniert. Dieses wird im folgenden Abschnitt beschrieben.

6.2.2 Funktionsweise

Um beliebige URLs in der eigenen Webapplikation zu ermöglichen, diese also von der Verzeichnisstruktur unabhängig zu gestalten, müssen alle Requests an einer Stelle zusammenlaufen, um sie dort zentral zu behandeln und zu entscheiden, ob es für diesen Pfad etwas anzuzeigen gibt, oder nicht. Dies ist ein bekanntes Prinzip in der Webapplikationsentwicklung, ein sogenanntes Entwurfsmuster (engl. „Design Pattern“) namens *Front Controller* [1, S. 344]. Dabei übernimmt ein Objekt – in PHP kann dies auch eine Datei sein, wenn Objektorientierung nicht zum Einsatz kommt – die Abarbeitung aller eingehenden Requests und entscheidet, welche Aktionen aufgerufen werden.

Ein Router in einer Webapplikation repräsentiert eben diesen Front Controller. Sein Code wird in der Regel in `index.php` im Root-Verzeichnis der Webapplikation platziert. Damit jedoch alle Requests – egal mit welcher Pfadangabe – dorthin umgeleitet werden, bedarf es zunächst einer Einstellung am Webserver.

Requests auf `index.php` umschreiben

Die hierfür benötigte Technologie nennt sich URL-Rewriting. Dabei werden mit Hilfe eines Webserver-Moduls die eingehenden Requests verändert, d. h. umgeschrieben.

Für den Apache Webserver – dieser kommt in den Docker-Containern zum Einsatz – muss dazu das Modul `mod_rewrite` aktiviert sein und für das Web-Verzeichnis die Direktive `AllowOverride All` gesetzt sein. Dies erlaubt, dass eine spezielle Datei namens `.htaccess` erstellt werden kann, die entsprechende Rewrite-Regeln enthält. Diese Datei wird dann ins Root-Verzeichnis der Webapplikation gelegt und sieht wie folgt aus:

```
1 <IfModule mod_rewrite.c>
2 RewriteEngine On
3
4 RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):~2$
5 RewriteRule ^(.*) - [E=BASE:%1]
```

```
6
7 RewriteCond %{REQUEST_FILENAME} !-f
8 RewriteRule ^ index.php [QSA,L]
9 </IfModule>
```

Zunächst wird überprüft, ob `mod_rewrite` aktiv ist und falls ja, die Rewrite-Engine aktiviert. In den Zeilen 4 und 5 wird dann das Basisverzeichnis ermittelt. Liegt das Projekt nämlich in einem Unterverzeichnis, muss dieses vor die anschließende Regel gesetzt werden. In Zeile 7 wird schließlich als Bedingung abgefragt, dass die Datei nicht existiert, in diesem Fall wird dann `index.php` in Zeile 8 aufgerufen. Das Argument `QSA` (Query String Append) gibt an, dass GET-Parameters angehängt werden, `L` (Last) stoppt die Verarbeitung von Regeln nach der aktuellen.

Die aktuelle Route ermitteln

Nachdem nun alle Requests auf `index.php` umgeschrieben wurden, kann die Verarbeitung in der PHP-Datei beginnen. Dazu werden nun zunächst die verwendete HTTP-Methode und der Pfad des Requests abgefragt. Beide Dinge sind im superglobalen Array `$_SERVER` vorhanden und können dort ausgelesen werden:

```
1 $method = $_SERVER["REQUEST_METHOD"];
2 $path = $_SERVER["REQUEST_URI"];
```

Die Methode ist ein String und in den einfachen Fällen „GET“ oder „POST“. Jeder einfache Request ist dabei ein GET-Request, POST kommt etwa beim Versenden von Formularen zum Einsatz. Es existieren auch noch weitere HTTP-Methoden, diese sind jedoch für diese Lehrveranstaltung noch nicht relevant und können etwa unter [3] nachgelesen werden.

Der Pfad ist ebenfalls ein String und enthält den gesamten URI ohne Domain- oder IP-Angabe. Bei `http://localhost:8080` ist dies einfach nur `/`, da es sich um das Root-Verzeichnis am Server (die oberste Ebene) handelt. Bei `http://localhost:8080/wet2vo-examples/vo06/routing/` ist dies `/wet2vo-examples/vo06/routing/`.

Hier zeigt sich nun schon das erste Problem: liegt die Applikation in einem oder mehreren Unterordnern, so gibt PHP diese natürlich mit aus. Die Routen sollen aber relativ zum Ordner des Projekts – in diesem Fall `routing` – sein und nicht zum Root-Verzeichnis. Daher ist die Einführung eines Basispfades (engl. „base path“) sinnvoll, der die Verzeichnisstruktur vom Root-Ordner bis zum Projektordner enthält. Mit ihm können später diese Teile des Pfades leicht entfernt werden.

```
1 $basePath = "/wet2vo-examples/vo06/routing";
```

Mit diesen Bestandteilen kann nun die Route gebildet werden. Sie besteht also aus Protokoll und Pfad. Der Basispfad wird dabei vorher mittels `substr()` entfernt, sodass nur noch der Relevante Teil übrig bleibt.

```
1 if (!empty($basePath)) {
2     $path = substr($path, strlen($basePath));
3 }
4
5 $route = "$method $path";
```

`$route` enthält nun also einen String mit Protokoll und Pfad. Für einen GET-Aufruf von `http://localhost:8080/wet2vo-examples/vo06/routing/` ist dies `GET /`, für einen GET-Aufruf von `http://localhost:8080/wet2vo-examples/vo06/routing/other` ist es `GET /other`.

Mit diesen Routen lässt sich nun das Verhalten des Front-Controllers steuern.

Aktionen für Routen ausführen

Im einfachsten Fall entscheidet nun ein `switch`-Statement, welcher Code ausgeführt wird. Verschiedene Fälle (`case`) enthalten Routen und den Code, der ausgeführt oder angezeigt werden soll. Im einfachsten Fall ist dies eine Ausgabe mit `echo`, es können aber genauso PHP-Dateien eingebunden (und somit ausgeführt) werden oder Latte-Templates angezeigt werden.

Trifft keine der Abfragen im `switch`-Statement zu, wird der `default`-Abschnitt ausgelöst. In ihm wird der HTTP-Statuscode 404 gesetzt und ausgegeben, dass keine Seite (oder Route) gefunden wurde. Damit ist die Abarbeitung beendet.

```
1 switch ($route) {  
2     case "GET /":  
3         echo "Here's the root page";  
4         break;  
5     case "GET /other":  
6         echo "Here's the other page.";  
7         break;  
8     default:  
9         http_response_code(404);  
10        echo "Page not found!";  
11 }
```

Auf dieselbe Art und Weise können beliebig viele Routen definiert werden, auch mit anderen Methoden wie etwa POST. Dieser gesamte Prozess ist in Abbildung 6.2 dargestellt.

Diese Art des Routings ist zwar simpel, jedoch nicht sehr flexibel, denn die Logik des Routings wird mit der eigenen Applikation vermischt. Aus diesem Grund empfiehlt es sich, bestehende Routing-Packages zu verwenden. Abschnitt 6.2.3 stellt das Paket `fhoee/router` vor, welches die gerade gezeigte Funktionalität objektorientiert umsetzt und für die Verwendung in dieser Lehrveranstaltung eine gute Basis darstellt. Für aufwändigere Ansprüche im Produktivbetrieb sind dann jedoch Router mit mehr Funktionalität gefordert; diese werden in Abschnitt 6.2.4 vorgestellt.

6.2.3 Einfaches Routing mit `fhoee/router`

`fhoee/router` ist eine PHP-Komponente, die das soeben beschriebene Prinzip des Routings objektorientiert umsetzt. Der Router wurde dabei bewusst für das Erlernen des Routings simpel gehalten und verfügt dabei nur über die grundsätzlichen Funktionen. Das Projekt ist open-source auf GitHub¹⁵ und über Packagist¹⁶ verfügbar.

¹⁵<https://github.com/Digital-Media/fhoee-router>

¹⁶<https://packagist.org/packages/fhoee/router>

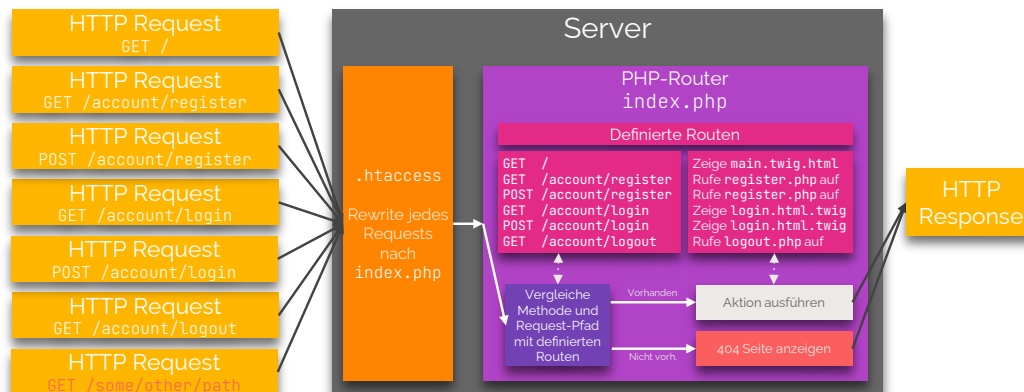


Abbildung 6.2: Das Prinzip des Routings. Alle Requests werden über die `.htaccess`-Datei auf `index.php` umgeleitet. Dort werden Methode und Pfad mit den im Router hinterlegten Routen verglichen. Gibt es eine Übereinstimmung, wird die bei der Route hinterlegte Aktion ausgeführt, die eine Response erzeugt. Gibt es keine Übereinstimmung, wird eine 404-Seite als Response zurückgegeben.

Verwendung und Ablauf von `fhoove/router`

Um `fhoove/router` im eigenen Projekt einzusetzen, muss die Komponente zunächst mittels Composer installiert werden.

```
1 $ composer require fhoove/router
```

Dies erzeugt die bereits in Vorlesung 5 vorgestellten Dateien `composer.json` und `composer.lock` sowie den Ordner `vendor`, in dem sich im Unterordner `fhoove/router/src` die Datei `Router.php` befindet. Sie enthält den gesamten Code des Routers.

Um den Router einzusetzen, wird nun im Projektordner eine Datei `index.php` angelegt und ebenfalls eine Datei `.htaccess` mit dem in Abschnitt 6.2.2 beschriebenen Inhalt. Letztere schreibt wiederum alle Requests auf `index.php` um. Ohne sie kann der gesamte Prozess nicht funktionieren.

`fhoove/router` bildet in der Klasse den in Abschnitt 6.2.2 beschriebenen Ablauf ab. Die Klasse kümmert sich also um das Erfragen der Methode und des Request-Pfades. Jedoch wird die Abfrage, ob dies einer definierten Route entspricht, nicht mehr wie gezeigt in einem `switch`-Statement abgehandelt. Vielmehr ermöglicht es die Klasse `Router`, Routen zu hinterlegen, die dann im Objekt gespeichert, bei der Ausführung verglichen und abgearbeitet werden. Die Arbeitsschritte sind also wie folgt:

1. Lege ein Router-Objekt an. Zuvor `autoload.php` aus dem `vendor`-Verzeichnis einbinden, damit die Klasse gefunden wird.
2. Optional: Setze einen Basispfad, falls `index.php` nicht auf der untersten (Root) Ebene des Webservers liegt, sondern in einem Unterverzeichnis.
3. Definiere alle gewünschten Routen und die dazugehörigen Aktionen, die eine Response erzeugen.
4. Definiere eine 404-Aktion, falls ein Pfad aufgerufen wird, für den es keine Route gibt.

5. Starte den Routing-Mechanismus.

Diese Schritte werden nun in den folgenden Abschnitten im Detail erläutert.

Ein Router-Objekt anlegen: Um ein Objekt von `fhoee/router` anzulegen, wird zunächst `autoload.php` aus dem `vendor`-Verzeichnis eingebunden. Somit ist das Autoloading von Composer aktiviert und alle benötigten Klassen werden automatisch gefunden. Danach wird mit `new` ein Objekt erzeugt. Der Konstruktor hat keine Parameter.

```
1 require __DIR__ . "/vendor/autoload.php";  
2  
3 $router = new Fhoee\Router\Router();
```

Um die Angabe des Namespaces bei der Objekt-Initialisierung zu vermeiden, kann alternativ ein `use`-Statement am Beginn der Datei eingesetzt werden. Die Initialisierung sieht dann wie folgt aus:

```
1 use Fhoee\Router\Router;  
2  
3 require __DIR__ . "/vendor/autoload.php";  
4  
5 $router = new Router();
```

Basispfad setzen: Befindet sich die Applikation in einem Unterverzeichnis des Webserver, so muss der Router diesen Pfad kennen. Im folgenden Szenario liegen die Dateien (also `index.php` und `.htaccess`) im Docker-Container unter `wet2vo-examples ▶ vo06 ▶ fhoee-router`. Dieser Teil des Verzeichnisses wird zum Basispfad. In Version 3.0 von `fhoee/router` wird dieser direkt über die öffentliche Eigenschaft `$basePath` gesetzt. Der Pfad muss dabei mit einem Schrägstrich (Slash) beginnen und darf am Ende keinen aufweisen:

```
1 $router->basePath = "/wet2vo-examples/vo06/fhoee-router";
```

Routen definieren: Der wohl wichtigste Schritt ist das Festlegen von Routen. Eine Route besteht im Falle der `fhoee/router`-Komponente aus drei Dingen:

1. Der HTTP-Methode (GET oder POST).
2. Dem Pfad, auch Muster (engl. „pattern“) genannt.
3. Einer Aktion, welche die Response dieser Route, d. h. Ausgabe, erzeugt. Diese Aktion wird in Form einer Callback-Funktion hinterlegt, die vom Router aufgerufen wird, wenn die Route zutrifft.

Durch `addRoute(HttpMethod $method, string $pattern, Closure $callback)` wird eine Route hinzugefügt. Die Methode enthält die drei genannten Teile als Parameter. `$method` ist vom Typ `HttpMethod`, ein Enum mit den Werten `HttpMethod::GET` und `HttpMethod::POST`. Der Typ `Closure`¹⁷ für den Callback repräsentiert dabei eine anonyme Funktion und bietet dabei etwas mehr Kontrolle als der ebenfalls verfügbare Typ `callable`, der auch normale Funktionen zulässt.

¹⁷<https://www.php.net/manual/de/class.closure.php>

Für die beiden in dieser Lehrveranstaltung relevanten HTTP-Methoden GET und POST existieren jedoch zwei Shorthand-Methoden, die die Angabe der Methode vereinfachen: `get(string $pattern, Closure $callback)` und zusätzlich der Shorthand `post(string $pattern, Closure $callback)`.

Mit ihnen können nun Routen definiert werden:

```
1 $router->get("/", function () {
2     require "views/main.php";
3 });
4
5 $router->get("/form", function () {
6     require "views/form.php";
7 });
8
9 $router->post("/form", function () {
10    require "views/form.php";
11 });
```

Diese Aufrufe definieren drei Routen:

- Eine GET-Route, die auf beim Aufruf des Root-Verzeichnisses des Projekts ausgelöst wird. Sie wird durch `/` repräsentiert und ergibt mit dem Basispfad dann den URL `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/`. Wird dieser URL aufgerufen, wird der Inhalt der Datei `views▶main.php` ausgegeben, die durch einen `require`-Aufruf eingebunden wird.
- Eine GET-Route, die beim Aufruf des Pfades `/form` ausgelöst wird. Sie wird durch den URL `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form` aufgerufen und zeigt ein Formularfeld (aus `views▶form.php`) an, welches eine Nachricht mit POST abschickt.
- Eine POST-Route, die beim Aufruf des Pfades `/form` ausgelöst wird. Sie wird ebenfalls durch `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form` aufgerufen, jedoch nur, wenn der Request mit POST, d. h. durch Absenden des Formulars, ausgelöst wurde.

Die Hauptdatei `views▶main.php` sieht wie folgt aus:

```
1 <?php
2
3 global $router;
4 ?>
5
6 <!DOCTYPE html>
7 <html lang="en">
8     <head>
9         <meta charset="UTF-8">
10        <title>Main Page</title>
11    </head>
12    <body>
13        <h1>Main</h1>
14        <p>This is the main page!</p>
15
16        <p>Why not also try the <a href="<?=$router->urlFor("/form") ?>">form</a>?</p>
17    </body>
18 </html>
```

Zu Beginn wird das Router-Objekt global eingebunden. Da die Datei `main.php` mit `require` in `index.php` eingebunden wird, kann das dort angelegte Objekt mittels `global`-Statement ganz einfach verfügbar gemacht werden.

Anschließend erfolgt die Ausgabe einer fast vollständig statischen HTML-Seite. Lediglich beim Link zum Formular findet sich PHP-Code: `$router->urlFor("/form")`. Die Methode `urlFor()` erhält als Argument eine Route – in diesem Fall `/form` – und löst diese in einen vollständigen URL auf, indem ein eventuell gesetzter Basispfad wieder davor gesetzt wird. Es entsteht also `http://localhost:8080//wet2vo-examples/vo06/fhooe-router/form`. Dies ist notwendig, denn würde nur `/form` als Wert des `href`-Attributs gesetzt, würde der Schrägstrich den Link ausgehend vom Root des Webserver setzen, wodurch `http://localhost:8080/form` entstünde, was nicht korrekt und gewünscht ist. Befindet sich die Applikation tatsächlich im Root des Webserver und nicht in einem Unterverzeichnis, so ist dies natürlich möglich.

Die Formulardatei `views/form.php` sieht ähnlich aus. Auch hier wird zunächst das Router-Objekt global eingebunden, um es bei URLs verwenden zu können.

```

1 <?php
2
3 global $router;
4 ?>
5
6 <!DOCTYPE html>
7 <html lang="en">
8   <head>
9     <meta charset="UTF-8">
10    <title>Form Page</title>
11  </head>
12  <body>
13    <h1>Form</h1>
14    <p>This is the form page!</p>
15
16    <form action="<?= $router->urlFor("/form") ?>" method="post">
17      <label for="message">Message:</label>
18      <input type="text" name="msg" id="message">
19      <button type="submit">Send Message</button>
20    </form>
21
22 <?php
23 if (isset($_POST["msg"])) {
24   echo "<p>Message: {$_POST["msg"]}</p>";
25 }
26 ?>
27
28   <p>Why not also try the <a href="<?= $router->urlFor("/") ?>">main page</a>?</p>
29 </body>
30 </html>

```

Dann wird ein Formularfeld eingebunden. Im `action`-Attribut kommt wiederum die Methode `urlFor()` zum Einsatz, da auch hier der korrekte URL aufgelöst werden muss. Die Ausgabe des Formularinhalts erfolgt nur dann, wenn sich auch Daten im superglobalen Array `$_POST["msg"]` befinden.

404 Callback setzen: Neben den Routen darf auch der Fehlerfall nicht außer Acht gelassen werden. Ruft der*die User*in eine Route auf, die nicht im Router hinterlegt wurde, muss eine Fehlerseite angezeigt werden. Dies geschieht wie folgt:

```
1 $router->set404Callback(function () {
2     require "views/404.html";
3 });
```

Die Methode `set404Callback(Closure $callback)` erwartet wiederum eine anonyme Funktion, die eine 404-Antwort zurückgibt. In diesem Fall wird eine statische HTML-Seite, nämlich `views/404.html` eingebunden und somit zurückgegeben:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>404</title>
6   </head>
7   <body>
8     <h1>404</h1>
9     <p>File not found.</p>
10  </body>
11 </html>
```

Die Klasse `Router` setzt dabei bereits selbst den HTTP Statuscode 404, dies muss also nicht extra geschehen.

Den Routing-Mechanismus starten: Nachdem nun im `Router`-Objekt alle notwendigen Informationen hinterlegt wurden, wird abschließend der Routing-Mechanismus gestartet:

```
1 $router->run();
```

Der Router beginnt nun den in Abbildung 6.2 dargestellten Ablauf. Er liest Methode und Pfad aus und vergleicht sie mit den zuvor hinterlegten Routen. Gibt es eine Übereinstimmung, so wird die hinterlegte Aktion aufgerufen und eine Response erzeugt. Wurden alle hinterlegten Routen überprüft und keine Übereinstimmung gefunden, wird der 404 Callback ausgelöst und diese Response generiert.

Weitere Funktionalitäten von `fhoee/router`

Im Folgenden werden noch weitere Methoden und Eigenschaften der `fhoee/router`-Klasse aufgelistet, die nützlich sein können:

public string \$basePath: Über diese Eigenschaft kann der Basispfad nicht nur gesetzt, sondern auch ausgelesen werden.

```
1 $basePath = $router->basePath;
```

public function redirect(string \$url, ?array \$par = null): void: Wird für eine Weiterleitung auf einen kompletten URL verwendet. Dadurch kann etwa auf eine externe Seite weitergeleitet werden. Mit dem optionalen Parameter `$par` können eventuelle GET-Parameter an den URL gehängt werden. Dabei wird automatisch der

Statuscode 302 Found gesetzt, der einen temporären Redirect anzeigt. Suchmaschinen können erkennen, dass es sich hier nicht um eine dauerhafte Weiterleitung handelt, die sie sich merken müssen.

```
1 $router->redirect("https://www.fh-ooe.at/");
```

`public function redirectTo(string $pattern): void:` Führt eine Weiterleitung auf einen Routingpfad durch. Dies ist sinnvoll für Redirects, die innerhalb der eigenen Seite geschehen. Der Pfad wird dabei zuvor in eine vollen URL aufgelöst und dann weitergeleitet.

```
1 $router->redirectTo("/form");
```

Basisprojekt fhooe/router-skeleton

Der soeben gezeigte Ablauf, mit dem die Komponente **fhooe/router** in ein eigenes Projekt integriert wird, ist universell einsetzbar. Die dabei notwendigen fünf Schritte sind jedoch immer dieselben. Aus diesem Grund, gibt es zum **fhooe/router** Paket ergänzend das **fhooe/router-skeleton**-Projekt. Es ist ebenfalls auf GitHub¹⁸ open-source verfügbar und über Packagist¹⁹ installierbar.

Da es sich nicht um eine Bibliothek (Library) handelt, sondern um ein Starter-Projekt, lautet der Composer-Aufruf hier anders:

```
1 $ composer create-project fhooe/router-skeleton path/to/install
```

`path/to/install` ist hierbei eine Pfadangabe, die noch nicht existieren darf. Der Pfad wird neu angelegt und das Starterprojekt wird dorthin installiert. Dies ist eigentlich nur ein verstecktes Git-Clone, weshalb auch der Pfad noch nicht existieren darf (denn Git klonet keine Repositories in bestehende Verzeichnisse).

Ausgehend vom Verzeichnis `wet2vo-examples▶vo06` wird somit das Unterverzeichnis **fhooe-router-skeleton** erzeugt. In ihm befinden sich alle zum Projekt gehörigen Dateien.

```
1 $ composer create-project fhooe/router-skeleton fhooe-router-skeleton
```

Nach dem Aufruf existieren in diesem Ordner die folgenden Unterverzeichnisse:

- **logs:** Verzeichnis für Applikations-Logdateien.
- **public:** Das Verzeichnis mit `index.php` und `.htaccess`. In einer Webserverkonfiguration für ein Produktivprojekt sollte **public** später das einzig öffentlich zugreifbare Verzeichnis sein.
- **views:** Verzeichnis mit den Views, also den Dateien, die bei den verschiedenen Routen angezeigt werden sollen. Dies können HTML-, PHP- oder auch Latte-Dateien sein.
- **vendor:** Verzeichnis mit allen benötigten Dateien wie dem **fhooe/router** oder dem **latte/latte** Paket.

¹⁸<https://github.com/Digital-Media/fhooe-router-skeleton>

¹⁹<https://packagist.org/packages/fhooe/router-skeleton>

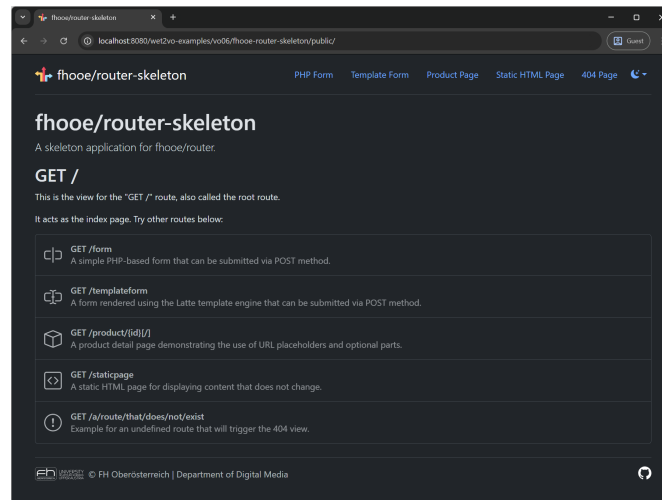


Abbildung 6.3: Die Startseite – auch als Root-Route bezeichnet – des `fhoove/router-skeleton` im dunklen Modus.

Damit das Projekt lauffähig ist, sollte zunächst in der Datei `public/index.php` der Basispfad in Zeile 49 gesetzt, d. h. angepasst werden. Das `public`-Verzeichnis muss hier mitbedacht werden.

```
1 $router->basePath = "/wet2vo-examples/vo06/fhoove-router-skeleton/public";
```

Danach kann dieser URL – hier `http://localhost:8080/wet2vo-examples/vo06/fhoove-router-skeleton/public/` im Browser geöffnet werden und zeigt die Startseite, wie in Abbildung 6.3 ersichtlich.

Sollte eine Fehlermeldung angezeigt werden, die etwas wie „Failed to open stream: Permission denied“ enthält, so darf Monolog aufgrund fehlender Berechtigungen nicht ins Verzeichnis `logs` schreiben. Dies geschieht meist auf Windows-Rechnern, bei denen Docker die Verzeichnisberechtigungen nicht korrekt mappen kann. In diesem Fall löst ein `chmod -R 777 fhoove-router-skeleton` das Problem.

Von hier aus können verschiedene bereits hinterlegte Routen ausprobiert und in `public/index.php` nachvollzogen werden:

- `GET /`: Zeige das Latte-Template `views/index.latte`.
- `GET /form`: Zeige ein Formular in der PHP-Datei `views/form.php`.
- `POST /form`: Zeige das Ergebnis des Formulars in der PHP-Datei `views/form.php`.
- `GET /templateform`: Zeige `views/templateform.latte`: ein Formular als Latte-Template.
- `POST /templateformresult`: Zeige das Ergebnis des Formulars im Latte-Template `views/templateformresult.latte`.
- `GET /product/{id}/[ ]`: Zeige die Seite `views/product.latte` für ein variables Produkt. Dabei ist `{id}` ein Platzhalter, der mit einem beliebigen Wert ersetzt und im Callback abrufbar ist. Der abschließende Schrägstrich ist durch das Setzen in eckige Klammern optional. Die Route funktioniert also mit und ohne.
- `GET /staticpage`: Zeige `views/staticpage.html`: eine statische HTML-Seite.

Latte-Templates verwenden: Der `fhooe/router-skeleton` definiert bereits alles Notwendige in `public/index.php`, um die Latte Template-Engine nahtlos mit dem Router zu verbinden. Die Latte-Engine wird instanziiert und das Verzeichnis für gecachte Templates gesetzt, die Template-Dateien selbst werden im Verzeichnis `views` erwartet. Weiters wird die Latte-Erweiterung `RouterExtension` hinzugefügt, welche die beiden Funktionen `url_for()` und `get_base_path()` definiert, damit diese auch in Template-Dateien verwendet werden können. Auch das superglobale Array `$_SESSION` (siehe Vorlesung 8) wird in den Templates durch die `SessionExtension` verfügbar gemacht.

```
1 $latte = new Engine();
2 $latte->setLoader(new FileLoader(__DIR__ . '/../views'));
3 $latte->setCacheDirectory(__DIR__ . '/../cache');
4 $latte->addExtension(new RouterExtension($router));
5 $latte->addExtension(new SessionExtension());
```

In einem Latte-Template kann danach beispielsweise der komplette URL zur Route `/form` ganz einfach über den Aufruf `{url_for("/form")}` dynamisch generiert werden.

Logging verwenden: Ebenfalls ist im Skeleton das Paket `monolog/monolog` eingebunden, welches einfach Logging-Aufruf in eine Datei absetzt. Das Paket wird ebenfalls in `public/index.php` initialisiert:

```
1 $logger = new Logger("skeleton-logger");
2 $logger->pushProcessor(new PsrLogMessageProcessor());
3 $formatter = new LineFormatter(
4     "[%datetime%] %channel%.%level_name%: %message%\n",
5     "d.m.Y H:i:s T",
6     true,
7     true,
8 );
9 $handler = new StreamHandler(__DIR__ . '/../logs/router.log");
10 $handler->setFormatter($formatter);
11 $logger->pushHandler($handler);
12
13 $router = new Router($logger);
```

Aufrufe des Loggers werden bereits in der Klasse `Router` vorgenommen und im Skeleton lediglich in die Datei weitergeleitet. Natürlich kann das `Logger`-Objekt auch jederzeit für eigene Logging-Aufruf verwendet werden:

```
1 $logger->info("Das ist eine Information.");
2 $logger->debug("Das ist eine Debug-Meldung.");
```

6.2.4 Komplexere Router

Wie bereits erwähnt, eignet sich `fhooe/router` für einfache Routing-Operationen und für das Erlernen des Prinzips, da er klein und übersichtlich gehalten wurde. Dies bedeutet aber selbstverständlich, dass der Funktionsumfang dementsprechend beschränkt ist. Darum sollte der Router auch nicht im Produktivbetrieb eingesetzt werden. Hierfür gibt es andere Komponenten.

bramus/router

Die Komponente **bramus/router** wurde von Bramus van Damme, einem Webentwickler aus Belgien entworfen. Das Projekt ist open-source auf GitHub²⁰ und auf Packagist²¹ und wird mit Composer installiert:

```
1 $ composer require bramus/router
```

Der Router ist ebenfalls einfach gehalten – er besteht nur aus einer Klasse – weist jedoch einen größeren Funktionsumfang auf. Folgende Dinge gehören dazu:

- Volle Unterstützung aller HTTP-Methoden wie GET, POST, PUT, DELETE, OPTIONS, PATCH und HEAD.
- Dynamische Routing-Muster (Pfade) über Regular Expressions.
- Untermuster für Routen, bei denen Teile optional gemacht werden können.
- Middleware Support (Code, der vor oder nach dem Routing ausgeführt werden kann).
- u. v. m.

Dieser Router eignet sich gut für kleine Projekte, wenn das eigene Wissen über Routing noch nicht allzu groß ist, denn viele der Dinge lassen sich gut im nur knapp über 500 Zeilen langen Kern nachvollziehen.

nikic/fast-route

Die Komponente **nikic/fast-route** wurde von Nikita Popov, einem Langzeit-Mitentwickler des PHP-Kerns, entworfen und stellt einen schnellen, auf regulären Ausdrücken basierenden Router dar. Das Projekt ist open-source auf GitHub²² und auf Packagist²³ verfügbar und wird über Composer installiert:

```
1 $ composer require nikic/fast-route
```

Der Fokus dieses Routers liegt auf Geschwindigkeit. Diese ist vor allem in umfangreichen Webapplikationen wichtig, wenn die Routing-Komponente möglicherweise hunderte verschiedene Routing-Muster zu verwalten hat. Wird nun der Routing-Mechanismus ausgelöst, so ist das Durchsuchen dieser vielen hinterlegten Routen möglicherweise ein langwieriger Prozess. **nikic/fast-route** löst dieses Problem mit Hilfe kombinierter Regular Expressions. Die Routen-Pfade werden als Regular-Expression hinterlegt, doch anstatt alle hinterlegten Muster einzeln zu durchlaufen, werden sie in ein großes Muster zusammengefügt und geklammert. Danach wird das `$matches`-Array durchlaufen: der erste Eintrag, der einen Wert enthält, ist die gesuchte Route. Nikita Popov beschreibt diesen Prozess ausführlich in einem Blog-Posting²⁴.

Die **nikic/fast-route** Komponente ist extrem umfangreich und zugleich performant, weshalb sie auch in vielen anderen Frameworks, wie etwa dem Slim Framework (siehe Vorlesung 12) eingesetzt wird. Sie eignet sich gut für große Projekte, das Verständnis von Routing sollte dementsprechend gut ausgeprägt sein.

²⁰<https://github.com/bramus/router>

²¹<https://packagist.org/packages/bramus/router>

²²<https://github.com/nikic/fastroute>

²³<https://packagist.org/packages/nikic/fast-route>

²⁴<https://www.npopov.com/2014/02/18/Fast-request-routing-using-regular-expressions.html>

league/route

Die Komponente `league/route` ist ein Router, der auf `nikic/fast-route` aufbaut und vollständige Unterstützung bzw. Implementierung vieler PSRs achtet. Dazu zählen PSR-4 (Autoloading), PSR-7 (HTTP Messages), PSR-11 (Container) und PSR-15 (HTTP Request Handler), PSR-16 (Caching) und PSR-17 (HTTP Factories). Sie ist open-source auf GitHub²⁵ und Packagist²⁶ verfügbar und wird mit Composer installiert:

```
1 $ composer require league/route
```

Die Ziele des Projekts bestehen darin, die performante `nikic/fast-route`-Komponenten benutzer*innenfreundlicher zu gestalten und durch strikte Einhaltung verschiedener PSRs bessere Kompatibilität und Austauschbarkeit in Projekten zu garantieren. Sie ist sicherlich derzeit die modernste PHP-Routing-Komponente, die sich für eine Vielzahl von Projekten eignet und mit dem entsprechenden Routing-Wissen eingesetzt werden kann.

6.3 Weiterführende Literatur

Zur Template-Engine Latte gibt es so gut wie keine brauchbare Literatur. Hier sei auf die offizielle Dokumentation [2] verwiesen, auch finden sich im Web zahlreiche Tutorials, welche die Abläufe erklären.

Auch zum Thema Routing ist die Literatur spärlich. In [4] wird in Kapitel 3 ein Router entworfen und dabei die Prinzipien erklärt. [5] greift das Prinzip des Front Controllers auf und demonstriert die Umsetzung anhand von PHP 8 Code. Ansonsten sei auf die diversen Ressourcen der jeweiligen Komponenten verwiesen, welche oft Einblicke in die Funktionsweise und das Design geben.

²⁵ <https://github.com/thephpleague/route>

²⁶ <https://packagist.org/packages/league/route>

Quellenverzeichnis

- [1] Martin Fowler. *Patterns of Enterprise Application Architecture*. Boston, USA: Addison Wesley, 1. Nov. 2002 (siehe S. 6-9).
- [2] David Grudl. *Latte Documentation*. Version 3.0. 27. Apr. 2026. URL: <https://latte.nette.org/en/guide> (siehe S. 6-8, 6-21).
- [3] Mozilla Developer Network. *HTTP request methods*. 4. Juli 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods> (besucht am 10.05.2026) (siehe S. 6-10).
- [4] Christopher Pitt. *Pro PHP 8 MVC. Model View Controller Architecture-Driven Application Development*. 2. Aufl. New York, USA: Apress, 27. Mai 2021 (siehe S. 6-21).
- [5] Matt Zandstra. *PHP 8 Objects, Patterns, and Practice: Volume 1. Mastering OO Enhancements and Design Patterns*. 7. Aufl. New York, USA: Apress, 2. Dez. 2024 (siehe S. 6-21).